HireX — Complete Test Case Repository

Document ID: `HXP-HANDOVER-02`
Platform Version: HireX V0 / Enterprise Multi-Tenant Recruitment Suite
Target Audience: QA Engineers, Manual & Automation Testers, Software Developers
Last Updated: October 2026

Table of Test Modules

1. Module 1: Authentication, Tenant Scoping & Session Security (`TC-AUTH-xxx`)
2. Module 2: Candidate Portal & Dynamic Application Wizard (`TC-CAND-xxx`)
3. Module 3: Recruiter Dashboard, Form Builder & Pipeline (`TC-REC-xxx`)
4. Module 4: Interviewer Portal, Availability & Scorecards (`TC-INT-xxx`)
5. Module 5: MeetX / Schedular Video Meetings & Webhooks (`TC-MEET-xxx`)
6. Module 6: Pipeline State Machine & Completion Guards (`TC-SM-xxx`)
7. Module 7: Document Verification & Compliance (`TC-DOC-xxx`)
8. Module 8: Branded Offer Letters & Expiration Lifecycle (`TC-OFF-xxx`)
9. Module 9: Job Feeds & Custom CNAME Domains (`TC-EXT-xxx`)
10. Module 10: End-to-End Critical Hiring Journeys (`TC-E2E-xxx`)

Module 1: Authentication, Tenant Scoping & Session Security

TC-AUTH-001: Candidate Registration with Password Validation

- **Module / Feature:** Authentication / Candidate Signup
- **Source Reference:** `hirex-Backend/src/candidates/candidate-auth.controller.ts`, `hirex-frontend/src/app/signup/page.tsx`
- **Objective:** Verify candidate account creation with password length and email validation.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Positive & Negative
- **Preconditions:** Unregistered candidate email address.
- **Role:** Candidate (Unauthenticated)
- **Test Procedure:**
  1. Navigate to candidate signup (`:3000/signup` or `https://dev.candidate.hire.zobay.in/signup`).
  2. Input name "Alice Candidate", email `alice.test@example.com`, and password `pass` (< 8 chars).

3. Attempt submission -> Observe validation error.
4. Input valid password `Password123!` and click "Sign Up".

- **Expected Results:**
  - Short password triggers inline client-side validation: "Password must be at least 8 characters long".
  - On valid submission, backend returns `201 Created` with user record.
  - Browser receives session cookie (`better-auth.session_token`).
  - User lands on Candidate Dashboard with welcome header.
- **Expected API Behavior:** `POST /api/auth/sign-up/email` returns `{ user: { id, email, role: 'CANDIDATE' } }`.
- **Database Side Effect:** `User` record created with `role = CANDIDATE`, `Account` record created with hashed password.
- **Actual Result:** Passed in automated test suite (`candidate-flow.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes (Supertest / Playwright)
- **Relevant Files:** `hirex-Backend/src/candidates/candidate-auth.controller.ts`, `hirex-frontend/src/app/signup/page.tsx`

TC-AUTH-002: Recruiter Registration with Company Tenant Creation

- **Module / Feature:** Authentication / Recruiter Onboarding
- **Source Reference:** `hirex-Backend/src/recruiters/recruiter-signup.controller.ts`, `hirex-employer-frontend/src/app/signup/page.tsx`
- **Objective:** Verify new organization creation and recruiter admin seat provisioning.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Integration
- **Preconditions:** Fresh work email.
- **Role:** Recruiter (Unauthenticated)
- **Test Procedure:**
  1. Open `:3002/signup` (`https://dev.recruiter.hire.zobay.in/signup`).
  2. Enter Company Name: "Apex Global Tech", Recruiter: "Marcus Recruiter", Work Email: `marcus@apextech.com`, Password: `Password123!`.
  3. Click "Create Recruiter Account".
- **Expected Results:**
  - Company tenant record created with slug generated from company name (`apex-global-tech`).
  - User assigned role `RECRUITER`, and `Recruiter` record created with `RecruiterRole = ADMIN`.
  - Recruiter dashboard loaded with company context initialized.

- **Expected API Behavior:** `POST /recruiters/signup` returns `201 Created` with company and recruiter details.
- **Database Side Effect:** `Company` created, `Recruiter` row created with foreign key `companyId`.
- **Actual Result:** Verified via automated integration test (`recruiter-auth.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/recruiters/recruiter-signup.controller.ts`

TC-AUTH-003: Interviewer Magic Link / Invite Token Redemption

- **Module / Feature:** Authentication / Interviewer Invitation
- **Source Reference:** `hirex-Backend/src/auth/interviewer-auth.controller.ts`, `interviewer/src/app/register/page.tsx`
- **Objective:** Verify token-based interviewer account activation and password setup.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Security
- **Preconditions:** Valid invitation token generated by recruiter for `interviewer.test@example.com`.
- **Role:** Interviewer (Invitee)
- **Test Procedure:**
  1. Open `:3003/register?token=<VALID_INVITE_TOKEN>`.
  2. Verify that company name and email are pre-filled and locked.
  3. Enter Name: "Devin Panelist", Password: `Password123!`.
  4. Submit activation form.
- **Expected Results:**
  - Invitation token verified from `Verification` table.
  - User credentials set, `Verification` token consumed and deleted (preventing replay attacks).
  - Interviewer authenticated and redirected directly to `/dashboard`.
- **Expected API Behavior:** `POST /auth/interviewer/invite/accept` returns `{ success: true, user: { ... } }`.
- **Actual Result:** Verified via automated test (`interviewer-auth.controller.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/auth/interviewer-auth.controller.ts`

TC-AUTH-004: Portal Session Isolation Guard (ADR-001)

- **Module / Feature:** Security / Portal Cross-Contamination Guard

- **Source Reference:** `hirex-Backend/src/auth/auth.guard.ts`, `hirex-Backend/src/auth/session-cookie.ts`
- **Objective:** Verify candidate session token cannot be used to execute recruiter actions.
- **Priority / Severity:** P0 / S1
- **Test Type:** Security, Negative
- **Preconditions:** Candidate logged in on `:3000` with active cookie.
- **Role:** Candidate attempting Recruiter access
- **Test Procedure:**
  1. Capture candidate session token cookie.
  2. Send `GET /recruiters/me` or navigate to `http://localhost:3002/dashboard`.
- **Expected Results:**
  - Backend `AuthGuard` extracts token, verifies role is `CANDIDATE`, and encounters `403 Forbidden` / `401 Unauthorized`.
  - Recruiter UI redirects to `/login` with clean state without leaking employer data.
- **Actual Result:** Verified via automated suite (`tenant-isolation.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes (Supertest)
- **Relevant Files:** `hirex-Backend/src/auth/auth.guard.ts`, `docs/ADR-001-portal-session-isolation.md`

Module 2: Candidate Portal & Dynamic Application Wizard

TC-CAND-001: Job Discovery & Multivariable Filter

- **Module / Feature:** Candidate Portal / Job Search
- **Source Reference:** `hirex-frontend/src/app/jobs/page.tsx`, `hirex-Backend/src/jobs/jobs.controller.ts`
- **Objective:** Ensure candidates can filter active job requisitions by keyword, location, and work mode.
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, UI
- **Preconditions:** At least 3 jobs published in database with varied attributes.
- **Role:** Candidate / Public Job Seeker
- **Test Procedure:**
  1. Navigate to `/jobs`.
  2. Type "Frontend" in search input.
  3. Select Location: "Bengaluru", Work Mode: "Remote".
- **Expected Results:**

    - Job card list updates dynamically to show matching jobs.
    - Non-matching jobs are filtered out.
    - Job card displays Title, Company Name, Location badge, and Salary range.
- **Actual Result:** Verified in manual test and component checks.
- **Status:** Pass
- **Automation Suitability:** Yes (Playwright)
- **Relevant Files:** `hirex-frontend/src/app/jobs/page.tsx`

TC-CAND-002: Dynamic Application Form Rendering & Validation

- **Module / Feature:** Candidate Application / Dynamic Wizard
- **Source Reference:** `hirex-frontend/src/app/jobs/[id]/apply/page.tsx`, `hirex-Backend/src/applications/applications.controller.ts`
- **Objective:** Ensure custom fields created in Employer Form Builder render correctly in candidate apply wizard.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Data-Driven
- **Preconditions:** Job configured with custom sections: "Work Experience" (Number) and "Visa Status" (Select).
- **Role:** Candidate
- **Test Procedure:**
    1. Click "Apply Now" on job detail page.
    2. In Step 2 (Screening & Custom Fields), inspect dynamically rendered questions.
    3. Leave mandatory custom field blank and click "Next".
    4. Fill mandatory field and click "Next".
- **Expected Results:**
    - Form dynamically creates input controls matching `QuestionType` (TEXT, NUMBER, SELECT, MULTI_SELECT).
    - Missing required fields block progression with inline error message.
    - Valid submission progresses to Step 3 (Resume & Review).
- **Actual Result:** Verified in unit test (`apply-job.dto.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-frontend/src/app/jobs/[id]/apply/page.tsx`

TC-CAND-003: Candidate Application Draft Auto-Save & Warning Guard

- **Module / Feature:** Candidate Portal / Draft Persistence
- **Source Reference:** `hirex-frontend/src/app/jobs/[id]/apply/page.tsx`
- **Objective:** Ensure candidate answers are saved locally to prevent data loss on accidental navigation.
- **Priority / Severity:** P1 / S2
- **Test Type:** UX, Local Storage
- **Preconditions:** Candidate filling multi-step application form.
- **Role:** Candidate
- **Test Procedure:**
  1. Fill Step 1 (Personal details) and Step 2 (Screening answers).
  2. Trigger browser reload (`F5`).
  3. Inspect form inputs.
  4. Attempt closing tab -> Observe browser dialog.
- **Expected Results:**
  - All previously entered field values are restored from `localStorage`.
  - Closing tab triggers native `beforeunload` unsaved changes prompt.
- **Actual Result:** Verified in commit `3bd007c`.
- **Status:** Pass
- **Automation Suitability:** Semi-automated (Playwright)
- **Relevant Files:** `hirex-frontend/src/app/jobs/[id]/apply/page.tsx`

TC-CAND-004: Self-Service Interview Slot Selection & Confirmation

- **Module / Feature:** Candidate Portal / Interview Booking
- **Source Reference:** `hirex-frontend/src/app/interviews/page.tsx`, `hirex-Backend/src/interviews/booking.service.ts`
- **Objective:** Allow candidate to select open interviewer slot and receive calendar booking.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Integration
- **Preconditions:** Application in `INTERVIEW` stage with generated slots.
- **Role:** Candidate
- **Test Procedure:**
  1. Open `/interviews` booking page.
  2. Select slot on calendar (e.g., Tomorrow at 2:00 PM).
  3. Click "Confirm Booking".
- **Expected Results:**

    - Slot status updates to `BOOKED`.
    - Interview status updates to `CONFIRMED`.
    - Candidate receives `.ics` calendar invite and candidate join URL (`candidateJoinUrl`).
- **Actual Result:** Verified in automated suite (`interviews.workflow.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/interviews/booking.service.ts`

TC-CAND-005: Atomic Concurrency Guard on Slot Booking

- **Module / Feature:** Interviews / Slot Booking Concurrency
- **Source Reference:** `hirex-Backend/src/interviews/booking.service.ts`
- **Objective:** Verify two candidates cannot book the exact same slot concurrently.
- **Priority / Severity:** P0 / S1
- **Test Type:** Concurrency, Race Condition
- **Preconditions:** 1 single open slot available for an interview.
- **Role:** Candidate A and Candidate B
- **Test Procedure:**
    1. Open slot booking on two separate sessions.
    2. Candidate A submits booking request.
    3. Candidate B submits booking request for same slot 50ms later.
- **Expected Results:**
    - Candidate A receives `200 OK` (Slot successfully booked).
    - Candidate B receives `409 Conflict` ("This slot has already been booked").
    - Database maintains strict 1-to-1 booking without slot duplication.
- **Actual Result:** Verified via automated test (`booking.concurrency.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes (Supertest concurrent requests)
- **Relevant Files:** `hirex-Backend/src/interviews/booking.service.ts`

Module 3: Recruiter Dashboard, Form Builder & Pipeline

TC-REC-001: Job Requisition with Custom Application Form Schema

- **Module / Feature:** Recruiter Portal / Job Creation & Form Builder

- **Source Reference:** `hirex-employer-frontend/src/app/jobs/new/page.tsx`, `hirex-Backend/src/jobs/jobs.controller.ts`
- **Objective:** Create and publish a job with custom screening sections.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Complex Data
- **Preconditions:** Authenticated recruiter admin.
- **Role:** Recruiter
- **Test Procedure:**
  1. Open `/jobs/new`.
  2. Fill title: "Staff Backend Architect", Department: "Engineering", Salary: "$150,000 - $180,000".
  3. Click "Add Custom Section" -> Name: "System Architecture Experience".
  4. Add Question: "How many years designing distributed message queues?" (Type: Number, Required: True).
  5. Click "Publish Job".
- **Expected Results:**
  - Job published with `status = OPEN`.
  - Custom form schema serialized as JSONB in `ApplicationForm` and `ApplicationQuestion` tables.
  - Job immediately discoverable on Candidate Portal.
- **Actual Result:** Verified in automated suite (`recruiters-job-form-transaction.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/jobs/jobs.controller.ts`

TC-REC-002: AI Resume Relevance Scoring (`aiScore`) Calculation

- **Module / Feature:** Recruiter Portal / AI Screening
- **Source Reference:** `hirex-Backend/src/common/utils/ai-match-scorer.ts`
- **Objective:** Verify candidate resume text is scored against job required skills.
- **Priority / Severity:** P1 / S2
- **Test Type:** Algorithm / NLP Unit Test
- **Preconditions:** Job requires `['NestJS', 'PostgreSQL', 'Docker']`.
- **Role:** System / Recruiter
- **Test Procedure:**
  1. Candidate submits resume listing "NestJS, PostgreSQL, Redis, Docker, Microservices".
  2. Inspect calculated `aiScore` in applicant profile.
- **Expected Results:**

    - `aiScore` calculates match percentage (e.g. 85–95%).
    - Matches are highlighted under "Matched Skills", missing under "Missing Skills".
    - If Groq API key is unset, falls back gracefully to local TF-IDF scorer without throwing exceptions.
- **Actual Result:** Verified via 8 unit test cases (`ai-match-scorer.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/common/utils/ai-match-scorer.ts`

TC-REC-003: Pipeline Stage Progression & Kanban Drag-and-Drop

- **Module / Feature:** Recruiter Dashboard / Applicant Kanban
- **Source Reference:** `hirex-employer-frontend/src/app/applicants/page.tsx` , `hirex-Backend/src/applications/applications.controller.ts`
- **Objective:** Move applicant through hiring stages visually.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, UI
- **Preconditions:** Applicant in `APPLIED` status.
- **Role:** Recruiter
- **Test Procedure:**
    1. Open applicant pipeline for job.
    2. Drag candidate card from `APPLIED` column to `SCREENING` column.
- **Expected Results:**
    - Card animates and snaps to `SCREENING` column.
    - Backend receives `PATCH /applications/:id/status` with `{ status: 'SCREENING' }` .
    - Candidate application status updates in database and reflects in Candidate portal timeline.
- **Actual Result:** Verified in automated test (`applications-global.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-employer-frontend/src/app/applicants/page.tsx`

Module 4: Interviewer Portal, Availability & Scorecards

TC-INT-001: Interviewer Recurring Weekly Availability Configuration

- **Module / Feature:** Interviewer Portal / Availability Management
- **Source Reference:** `interviewer/src/app/availability/page.tsx` , `hirex-Backend/src/interviews/availability.controller.ts`

- **Objective:** Configure recurring available hours for candidate interview bookings.
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, Calendar
- **Preconditions:** Authenticated interviewer.
- **Role:** Interviewer
- **Test Procedure:**
  1. Open `/availability` in Interviewer Portal.
  2. Enable Tuesday & Thursday, 14:00 to 18:00. Timezone: `Asia/Kolkata`.
  3. Click "Save Availability".
- **Expected Results:**
  - Recurring availability saved in `InterviewAvailability` table.
  - Recruiter slot generator uses these windows to compute candidate booking slots.
- **Actual Result:** Verified via automated test (`availability.service.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/interviews/availability.controller.ts`

TC-INT-002: Private Interviewer Briefing vs Public Candidate Agenda

- **Module / Feature:** Interviewer Portal / Information Security
- **Source Reference:** `hirex-Backend/src/interviews/interviews.service.ts`
- **Objective:** Verify private recruiter briefing notes are never leaked to the candidate API responses.
- **Priority / Severity:** P0 / S1
- **Test Type:** Security, Data Masking
- **Preconditions:** Interview record with `candidateNotes` and `interviewerNotes`.
- **Role:** Candidate & Interviewer
- **Test Procedure:**
  1. Fetch interview via Interviewer API (`GET /interviews/:id`).
  2. Fetch same interview via Candidate API (`GET /candidate/applications/:id`).
- **Expected Results:**
  - Interviewer response includes `interviewerNotes` (e.g. "Candidate was weak on concurrency in round 1; test deep deadlock questions").
  - Candidate response strips `interviewerNotes` and contains only `candidateNotes`.
- **Actual Result:** Verified via automated test (`interviews.auth.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes

- **Relevant Files:** `hirex-Backend/src/interviews/interviews.service.ts`

TC-INT-003: Structured Scorecard & Cultural Fit Score Submission

- **Module / Feature:** Interviewer Portal / Scorecards
- **Source Reference:** `interviewer/src/app/interviews/[id]/feedback/page.tsx`, `hirex-Backend/src/interviews/interviews.controller.ts`
- **Objective:** Record rubric ratings, cultural fit score, and recommendation.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, Validation
- **Preconditions:** Interview in `CONFIRMED` status.
- **Role:** Interviewer
- **Test Procedure:**
  1. Open interview scorecard page.
  2. Rate Technical Skills: 4/5, Communication: 5/5, Problem Solving: 4/5.
  3. Rate Cultural Fit Score: 5/5.
  4. Select Recommendation: `HIRE`.
  5. Add qualitative summary and click "Submit Feedback".
- **Expected Results:**
  - `InterviewFeedback` record created with ratings JSONB and `culturalFitScore = 5`.
  - Interview status transitions to `COMPLETED`.
  - Recruiter dashboard updates with scorecard summary badge.
- **Actual Result:** Verified via automated test (`interviews.workflow.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/interviews/interviews.controller.ts`

TC-INT-004: Edit and Update Existing Scorecard

- **Module / Feature:** Interviewer Portal / Scorecard Modification
- **Source Reference:** `interviewer/src/app/interviews/[id]/feedback/page.tsx`
- **Objective:** Allow an interviewer to update previously submitted feedback without duplicate key violations.
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, Upsert
- **Preconditions:** Existing submitted scorecard on interview.
- **Role:** Interviewer

- **Test Procedure:**
  1. Re-open completed interview evaluation view.
  2. Adjust Technical rating from 4 to 5 and add updated summary note.
  3. Click "Update Feedback".
- **Expected Results:**
  - System executes an `upsert` or `update` on `InterviewFeedback`.
  - Returns `200 OK`; updated ratings persist without throwing database unique constraint errors.
- **Actual Result:** Verified in commit `93279ab`.
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/interviews/interviews.service.ts`

Module 5: MeetX / Schedular Video Meetings & Webhooks

TC-MEET-001: Schedular Meeting Creation with Dev Mock Fallback

- **Module / Feature:** MeetX Integration / Non-Production Mock
- **Source Reference:** `hirex-Backend/src/interviews/providers/schedular-meeting.provider.ts`
- **Objective:** Ensure safe offline development when `SCHEDULAR_API_KEY` is not configured.
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, Fallback
- **Preconditions:** `NODE_ENV=development`, `SCHEDULAR_API_KEY=""`.
- **Role:** System / Booking Service
- **Test Procedure:**
  1. Candidate books an interview slot.
  2. Check generated meeting link on interview record.
- **Expected Results:**
  - Backend logs a development warning.
  - Generates meeting link in format: `https://meet.hirex.dev/join/mtg_[random8]`.
  - Booking completes successfully with 200 OK.
- **Actual Result:** Verified in manual test and provider inspection.
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/interviews/providers/schedular-meeting.provider.ts`

TC-MEET-002: Live Schedular API Meeting Creation & Privilege URL Split

- **Module / Feature:** MeetX Integration / Live Video Provider
- **Source Reference:** `hirex-Backend/src/interviews/providers/schedular-meeting.provider.ts`
- **Objective:** Verify live meeting creation splits host vs attendee privileges.
- **Priority / Severity:** P0 / S1
- **Test Type:** Integration, External API
- **Preconditions:** `SCHEDULAR_API_KEY` configured in `.env`.
- **Role:** System
- **Test Procedure:**
  1. Execute `SchedularMeetingProvider.createMeeting(...)`.
  2. Inspect response object.
- **Expected Results:**
  - Dispatches POST request to `https://api.dev.scheduler.zobay.in/api/v1/meetings`.
  - Receives distinct `hostJoinUrl` (privileged room controls) and `candidateJoinUrl` (attendee controls).
  - Both URLs saved to `interviews` table.
- **Actual Result:** Verified in provider unit tests (`schedular-meeting.provider.ts`).
- **Status:** Pass
- **Automation Suitability:** Mocked in automated suite
- **Relevant Files:** `hirex-Backend/src/interviews/providers/schedular-meeting.provider.ts`

TC-MEET-003: Webhook Auto-Completion (`meeting.ended`)

- **Module / Feature:** Webhooks / Interview Lifecycle Sync
- **Source Reference:** `hirex-Backend/src/webhooks/schedular-webhook.service.ts`
- **Objective:** Mark interview `COMPLETED` when video call terminates.
- **Priority / Severity:** P0 / S1
- **Test Type:** Integration, Webhook
- **Preconditions:** Interview with `meetingId = 'mtg_sched_89123'` in `CONFIRMED` status.
- **Role:** Webhook Ingestion
- **Test Procedure:**
  1. Send POST to `/webhooks/schedular` with payload: `{ "event": "meeting.ended", "meetingId": "mtg_sched_89123" }` and valid HMAC signature.
- **Expected Results:**
  - Signature validated against `SCHEDULAR_WEBHOOK_SECRET`.

    - Interview status transitions from `CONFIRMED` to `COMPLETED`.
    - Returns `200 OK` with `{ received: true }`.
- **Actual Result:** Verified via automated test (`schedular-webhook.controller.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/webhooks/schedular-webhook.service.ts`

TC-MEET-004: Fail-Closed Webhook Security on Tampered Signatures

- **Module / Feature:** Webhooks / Security
- **Source Reference:** `hirex-Backend/src/webhooks/schedular-webhook.controller.ts`
- **Objective:** Reject unsigned or invalid webhook payloads.
- **Priority / Severity:** P0 / S1
- **Test Type:** Security, Negative
- **Preconditions:** Backend running with active webhook secret.
- **Role:** Unauthenticated / Attacker
- **Test Procedure:**
    1. Send POST to `/webhooks/schedular` with invalid `x-signature` header or mismatched meeting ID.
- **Expected Results:**
    - Request rejected with `401 Unauthorized` or `400 Bad Request`.
    - Database interview state remains unchanged.
- **Actual Result:** Verified via automated test (`schedular-webhook.controller.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/webhooks/schedular-webhook.controller.ts`

Module 6: Pipeline State Machine & Completion Guards

TC-SM-001: Strict Reversion Guard (No Reversion to `APPLIED`)

- **Module / Feature:** Recruitment Pipeline / State Machine Integrity
- **Source Reference:** `hirex-Backend/src/applications/applications.service.ts`
- **Objective:** Prevent candidate cards from regressing to initial `APPLIED` status.
- **Priority / Severity:** P0 / S1
- **Test Type:** Business Logic, Negative

- **Preconditions:** Candidate in `SCREENING` or `INTERVIEW` stage.
- **Role:** Recruiter
- **Test Procedure:**
  1. Send `PATCH /applications/:id/status` with `{ "status": "APPLIED" }`.
- **Expected Results:**
  - Backend validation rejects request with `400 Bad Request` ("Cannot revert candidate to APPLIED stage").
  - In Recruiter Kanban, dragging card back to Applied column is disabled or snaps back.
- **Actual Result:** Verified via automated test (`applications-global.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/applications/applications.service.ts`

TC-SM-002: Open Interview Hard Block on Pipeline Advancement

- **Module / Feature:** Recruitment Pipeline / Stage Guards
- **Source Reference:** `hirex-Backend/src/applications/applications.service.ts`
- **Objective:** Enforce that all open rounds must finish before an offer can be extended.
- **Priority / Severity:** P0 / S1
- **Test Type:** Business Logic, Guardrail
- **Preconditions:** Application in `INTERVIEW` stage with 1 round in `CONFIRMED` status.
- **Role:** Recruiter
- **Test Procedure:**
  1. Recruiter clicks "Advance to Offer Stage".
- **Expected Results:**
  - System blocks stage transition with error: "Cannot advance application to Offer stage while interviews are open or pending feedback".
  - Candidate remains in `INTERVIEW` stage until scorecard is submitted.
- **Actual Result:** Verified via automated test (`interviews.workflow.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/applications/applications.service.ts`

TC-SM-003: Post-Offer Acceptance Document Verification Gate

- **Module / Feature:** Recruitment Pipeline / Compliance Transition
- **Source Reference:** `hirex-Backend/src/candidates/candidates.service.ts`

- **Objective:** Verify candidate offer acceptance shifts application directly to `DOCUMENT_VERIFICATION`.
- **Priority / Severity:** P0 / S1
- **Test Type:** State Machine, Positive
- **Preconditions:** Application in `OFFER_STAGE` with `EXTENDED` offer.
- **Role:** Candidate
- **Test Procedure:**
  1. Candidate signs and accepts offer.
- **Expected Results:**
  - Offer status becomes `ACCEPTED`.
  - Application status automatically advances to `DOCUMENT_VERIFICATION`.
  - Cannot jump directly to `HIRED`.
- **Actual Result:** Verified in PRD gap closure migration and tests (`20261023_prd_gap_closure`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/candidates/candidates.service.ts`

TC-SM-004: Finalize Hire Gate Enforcement

- **Module / Feature:** Recruitment Pipeline / Final Hire Gate
- **Source Reference:** `hirex-employer-frontend/src/app/applicants/[applicationId]/page.tsx`
- **Objective:** Ensure moving to `HIRED` is locked until all required documents are verified.
- **Priority / Severity:** P0 / S1
- **Test Type:** Compliance, Guardrail
- **Preconditions:** Application in `DOCUMENT_VERIFICATION` with 1 document unverified.
- **Role:** Recruiter
- **Test Procedure:**
  1. Inspect status dropdown -> `HIRED` option is locked/disabled.
  2. Attempt clicking "Finalize Hire".
- **Expected Results:**
  - System flags unverified documents ("2 of 3 required documents are pending verification").
  - "Finalize Hire" button is disabled until all documents are marked `VERIFIED`.
- **Actual Result:** Verified in commit `54441e2`.
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-employer-frontend/src/app/applicants/[applicationId]/page.tsx`

Module 7: Document Verification & Compliance

TC-DOC-001: Multi-File Document Upload & Cloudflare R2 Storage

- **Module / Feature:** Documents / File Storage
- **Source Reference:** `hirex-Backend/src/documents/documents.service.ts`
- **Objective:** Allow candidate to upload multi-file documents (e.g., front/back ID).
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, File I/O
- **Preconditions:** Application in `DOCUMENT_VERIFICATION`.
- **Role:** Candidate
- **Test Procedure:**
  1. Open `/candidate/documents`.
  2. Select "Government ID Proof".
  3. Upload `id_front.jpg` and `id_back.jpg`.
  4. Submit upload.
- **Expected Results:**
  - Files streamed to Cloudflare R2 bucket (`hirex-test-dev`).
  - Two `DocumentFile` records created linked to parent `Document`.
  - Status sets to `PENDING`.
- **Actual Result:** Verified via automated test (`documents.service.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/documents/documents.service.ts`

TC-DOC-002: Document Rejection & Candidate Re-Upload Loop

- **Module / Feature:** Documents / Audit Trail
- **Source Reference:** `hirex-Backend/src/documents/documents.controller.ts`
- **Objective:** Verify recruiter document rejection requires a reason and candidate re-upload clears audit stamps.
- **Priority / Severity:** P1 / S2
- **Test Type:** Functional, Workflow
- **Preconditions:** Uploaded document in `PENDING` state.
- **Role:** Recruiter & Candidate
- **Test Procedure:**

1. Recruiter clicks "Reject Document" on illegible file without entering reason -> Blocked.
2. Recruiter enters reason "Passport expiration date blurred" and submits.
3. Candidate views document in candidate portal -> Note displayed.
4. Candidate uploads replacement file.

- **Expected Results:**
  - Rejection requires non-empty reason string.
  - On candidate replacement upload, `rejectionReason`, `verifiedAt`, and `verifiedBy` are cleared.
  - Status resets to `PENDING` for recruiter re-review.
- **Actual Result:** Verified via automated test (`documents.controller.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/documents/documents.controller.ts`

Module 8: Branded Offer Letters & Expiration Lifecycle

TC-OFF-001: Template Placeholder Interpolation & PDF Generation

- **Module / Feature:** Offers / PDF Engine
- **Source Reference:** `hirex-Backend/src/offers/offers.service.ts`
- **Objective:** Interpolate all placeholders and generate downloadable PDF.
- **Priority / Severity:** P0 / S1
- **Test Type:** Functional, PDF
- **Preconditions:** Candidate in `OFFER_STAGE`.
- **Role:** Recruiter
- **Test Procedure:**
  1. Open Offer modal.
  2. Set Base Salary: "$140,000", Start Date: "2026-11-15", Expiration: "7 days".
  3. Generate offer.
- **Expected Results:**
  - Replaces `{{candidate_name}}`, `{{job_title}}`, `{{company_name}}`, `{{salary}}`, `{{start_date}}`, `{{expiry_date}}`.
  - PDF document rendered without unparsed `{{...}}` tags.
  - File persisted to Cloudflare R2 and attached to `Offer` record.
- **Actual Result:** Verified via automated test (`offers.service.spec.ts`).
- **Status:** Pass

- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/offers/offers.service.ts`

TC-OFF-002: Stale Offer Auto-Expiration Cron Job

- **Module / Feature:** Offers / Scheduled Tasks
- **Source Reference:** `hirex-Backend/src/offers/offers.service.ts`
- **Objective:** Verify hourly cron job marks expired offers as `EXPIRED`.
- **Priority / Severity:** P1 / S2
- **Test Type:** Scheduled Task / Background Worker
- **Preconditions:** Offer in `EXTENDED` status where `expiresAt` < current time.
- **Role:** System Background Worker
- **Test Procedure:**
  1. Trigger offer expiration routine (`@Cron('0 * * * *')`).
- **Expected Results:**
  - Offer status transitions from `EXTENDED` to `EXPIRED`.
  - Candidate viewing offer sees expiration banner and "Accept" CTA is disabled.
- **Actual Result:** Verified via unit test (`offers.service.spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/offers/offers.service.ts`

Module 9: Job Feeds & Custom CNAME Domains

TC-EXT-001: XML Job Distribution Feeds for Aggregators

- **Module / Feature:** Integrations / Job Boards
- **Source Reference:** `hirex-Backend/src/recruiter/integrations/feeds.controller.ts`
- **Objective:** Verify standard XML feeds for Indeed, LinkedIn, and Naukri.
- **Priority / Severity:** P1 / S2
- **Test Type:** Integration, Feed Syndication
- **Preconditions:** At least 1 published job in the system.
- **Role:** Public Aggregator
- **Test Procedure:**
  1. Send `GET /feeds/indeed-jobs.xml`.
  2. Send `GET /feeds/linkedin-jobs.xml`.
  3. Send `GET /feeds/naukri-jobs.xml`.

- **Expected Results:**
  - Returns `200 OK` with `Content-Type: application/xml` or `text/xml`.
  - Contains valid XML tags (`<job>` , `<title>` , `<company>` , `<location>` , `<url>`).
- **Actual Result:** Verified via automated test (`feeds.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-Backend/src/recruiter/integrations/feeds.controller.ts`

TC-EXT-002: Custom Career Page CNAME Reverse Proxy Resolution

- **Module / Feature:** Multi-Tenancy / Custom Domains
- **Source Reference:** `hirex-frontend/src/proxy.ts` , `hirex-Backend/src/career-page/career-page.controller.ts`
- **Objective:** Verify foreign host headers rewrite directly to company branded career page.
- **Priority / Severity:** P1 / S2
- **Test Type:** Network / Proxy Routing
- **Preconditions:** Company configured with `careerPageCustomDomain = 'careers.democorp.com'` .
- **Role:** Job Seeker visiting Custom Domain
- **Test Procedure:**
  1. Send request to Candidate Portal with header `Host: careers.democorp.com` .
- **Expected Results:**
  - `src/proxy.ts` queries `/public/career-domain?host=careers.democorp.com` .
  - Resolves company slug and rewrites path to `/careers/hirex-demo-corp/...` .
  - Displays company logo, banner, and published jobs without HireX platform branding.
- **Actual Result:** Verified in proxy tests and PRD gap closure notes.
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** `hirex-frontend/src/proxy.ts`

Module 10: End-to-End Critical Hiring Journeys

TC-E2E-001: Complete Candidate-to-Hire Golden Path

- **Module / Feature:** Full Platform / Multi-Role Workflow
- **Source Reference:** All Repositories
- **Objective:** Validate complete lifecycle from job requisition to hired status across all 3 portals.
- **Priority / Severity:** P0 / S1

- **Test Type:** End-to-End User Journey
- **Preconditions:** Fresh tenant database or clean organization space.
- **Roles:** Recruiter, Candidate, Interviewer
- **Test Procedure:**
  1. **Recruiter** publishes "Senior Cloud Architect" with custom form.
  2. **Candidate** discovers job, fills 3-step wizard with resume, and submits.
  3. **Recruiter** reviews candidate card, verifies `aiScore`, and advances to `INTERVIEW`.
  4. **Interviewer** sets weekly availability.
  5. **Candidate** self-books tomorrow at 3:00 PM -> MeetX room generated.
  6. **Interviewer** joins call, reviews private briefing, and submits scorecard (`HIRE`, Cultural Fit: 5/5).
  7. **Recruiter** generates PDF offer and extends it with 7-day expiration.
  8. **Candidate** signs and accepts offer -> moves to `DOCUMENT_VERIFICATION`.
  9. **Candidate** uploads ID proof and university degree.
  10. **Recruiter** inspects files in-browser and marks them `VERIFIED`.
  11. **Recruiter** clicks "Finalize Hire".
- **Expected Results:**
  - Every stage transition executes cleanly without errors.
  - Final candidate status is `HIRED`.
  - All audit stamps (`verifiedBy`, `verifiedAt`, `createdAt`) are recorded in PostgreSQL.
- **Actual Result:** Verified end-to-end via automated suite (`candidate-flow.e2e-spec.ts`).
- **Status:** Pass
- **Automation Suitability:** Yes
- **Relevant Files:** All controllers, services, and frontend pages.